`subselect` — Methodology Log

Athanasios Tsilimigkras
Technical University of Crete, Chania, Greece

Last updated: May 3, 2026

Abstract

This document records the finalized methodology decisions of the `subselect` framework for country-scale CMIP6 sub-ensemble selection. Each decision lists the question, the alternatives considered, the chosen option, the rationale, the defining equation or pseudocode, the implementation pointer, and the date. The document is appended to as decisions are made, never edited retroactively. It feeds the EGU 2026 poster (abstract EGU26-14502) and the follow-up framework paper.

Contents

1 Historical performance **1**
1.1 Data sources and regridding . . . 1
1.2 Observation reference grid for σ_{obs} vs per-pixel metrics . . . 2
1.3 Bias-map observed-mean panel: native 0.5° W5E5 . . . 3
1.4 Methodology corrections (post-paper) . . . 3

2 Future-response spread **4**
2.1 Change-signals regression-test tolerance . . . 4

3 Model independence **5**

4 Multi-objective optimization **5**

5 Country cropping **5**
5.1 Framework default crop rule . . . 5

6 Caching and reproducibility **6**
6.1 Cache stack: parquet + zarr + SQLite . . . 6

1 Historical performance

The historical-performance dimension is reused unchanged from the published Greece paper. See `../docs/historical_performance.md` for the complete specification of the variables, reference dataset, evaluation window, and aggregation choices.

1.1 Data sources and regridding

Question. Which CMIP6 grid and which observation product back the published Greece paper's HPS values, and what regridding choices stand behind them?

Alternatives considered.

- Among the precomputed CMIP6 products on disk (`monthly`, `monthly_new`, `monthly_original`; pre-restructure names): each was a separate ingest with its own time-conventions and decoding choices.
- Among the observation products: raw ISIMIP3a (`ISIMIP3a/monthly/`, native ∼0.5° grid), single-grid upscaled (`ISIMIP3a/monthly_upscaled/`, common grid for all models), per-CMIP6-model upscaled (`ISIMIP3a/monthly_cmip6_upscaled/<MODEL>/`, model-by-model grid).
- Among regridding schemes: `xesmf` bilinear, `xesmf` conservative, scipy nearest-neighbor, `cdo` remapcon / remapnn / remapbil.

Choice. CMIP6 from `Data/CMIP6/monthly/` (post-restructure of `monthly_new/`) on each model's native grid; observations from `Data/reference/monthly_cmip6_upscaled/<MODEL>/` for the per-pixel model-vs-obs metrics; observations regridded with a per-variable scheme that is *conservative interpolation for* `pr` and *nearest-neighbor for* `tas` / `psl` / `tasmax` (per the inline comment in `legacy/cmip6-greece/GR_model_performance_HM.ipynb` line 55). The 1995–2014 evaluation window per `Config.eval_window` is sliced *after* loading.

Rationale. The choices are inherited from the published Greece paper (Tsilimigkras et al., 2026) and verified by the regression test, not re-litigated. The pipeline does *no* on-the-fly regridding; regridding is upstream pre-processing performed during the ISIMIP3a ingest, and `subselect` consumes the artefacts as-is. The tooling that produced the upscaled files is not part of the framework's dependency surface.

Implementation. `subselect.io.load_cmip6` reads `Data/CMIP6/monthly/<var>/<scenario>/<MODEL>_*.nc`; `subselect.io.load_w5e5` reads `Data/reference/monthly_cmip6_upscaled/<MODEL>/<var>_*_<MODEL>.nc` via `REFERENCE_FILENAME_TEMPLATES`. Period slicing happens inside `subselect.performance.compute_hps`.

Decided. 2026-05-01, in version 0.1.0.

1.2 Observation reference grid for σ_{obs} vs per-pixel metrics

Question. Which observation reference grid is used for the σ_{obs} scalar in the TSS denominator, versus the per-pixel bias / RMSE / correlation / bias_score metrics that compare a model to observations grid-cell-by-grid-cell?

Alternatives considered.

- **Both metrics from the per-CMIP6-model upscaled product.** One σ_{obs} value per (variable, season, *model*), regridded variance bakes into TSS.
- **Both metrics from a single common grid.** One σ_{obs} per (variable, season), but per-pixel comparisons are no longer on each model's native grid — model fields would need to be regridded to the common grid, which the framework explicitly avoids.
- **Two-tier reference (chosen).** σ_{obs} from a single common grid; per-pixel metrics from each model's own grid.

Choice. Two-tier reference: σ_{obs} from the single-grid upscaled product (`Data/reference/monthly_upscaled/<var>_*_cmip6_upscaled.nc`), per-pixel metrics from the per-CMIP6-model upscaled product (`Data/reference/monthly_cmip6_upscaled/<MODEL>/<var>_*_<MODEL>.nc`).

Rationale. Decouples TSS values from model-specific regridding variance. Per-model σ_{obs} would couple TSS to the idiosyncrasies of each model's grid (cell sizes, $\cos\varphi$ mass distribution, regridding artefacts at coastlines), so the cross-model min–max normalisation that feeds HMperf would be measuring a mixture of *model fidelity* and *regridding fidelity*. A single common σ_{obs} removes that confound and preserves cross-model comparability for the downstream normalisation. The per-pixel comparison metrics (bias, RMSE, correlation, bias_score) must stay on each model's own grid because they compare *model field* to *obs field* cell by cell, and the framework does not regrid models. The two-product setup costs the storage of one extra reference dataset on disk and a second loader path in `subselect.io`.

Initial parity caveat (corrected in 0.1.0). The published Greece pipeline reduced the single-grid per-pixel std-over-months map to σ_{obs} via an *unweighted* `.mean(dim=['lat', 'lon'])`, inconsistent with the $\cos\varphi$-weighted spatial mean used everywhere else. Version 0.1.0 restored the $\cos\varphi$ weighting; see § Methodology corrections (post-paper).

Implementation. `subselect.io.load_single_grid_w5e5` (one file per variable, common grid) for σ_{obs}; `subselect.io.load_w5e5` (per-model upscaled) for the per-pixel comparison. `Config.single_grid_reference_root` (default `<data_root>/reference/monthly_upscaled`) and `Config.reference_root` (default `<data_root>/reference/monthly_cmip6_upscaled`) keep the two products separately addressable; both are TOML-overridable.

Decided. 2026-05-01, in version 0.1.0. Both initial parity caveats (the unweighted spatial mean and the cmip6-grid upscaled σ_{obs} source) were corrected in version 0.1.0; see § Methodology corrections (post-paper).

1.3 Bias-map observed-mean panel: native 0.5° W5E5

The bias-map figure (`subselect.viz.performance_figs.fig_bias_maps_per_variable`) has a top-row *observed-mean* panel that displays the W5E5 reference field at high resolution for visual context, separate from the per-model bias panels below it. The observed-mean panel is read from the native 0.5° W5E5 product (360 × 720), not from the cmip6-grid upscaled product used for σ_{obs}.

The bias-map adapter (`subselect.viz._data_adapters.build_bias_maps_state`) populates `observed_maps[period][variable]` via `subselect.io.load_native_w5e5`, which resolves to `<data_root>/reference/monthly_05/<var>_*.nc`. This yields the high-resolution observed-mean panel that pairs naturally with the per-model bias maps (~520 native pixels in the Greece bbox, vs the ~49 pixels the cmip6-grid upscaled product would have produced). Per-model bias panels remain unchanged — they use the per-CMIP6-model upscaled obs (`load_w5e5`) since the bias is computed cell-by-cell on each model's own grid.

Implementation. `subselect.io.load_native_w5e5(variable)` returns the native 0.5° dataset; the loader sorts by lat/lon on read because ISIMIP3a files are stored with descending latitude. Decided 2026-05-01, in version 0.1.0.

1.4 Methodology corrections (post-paper)

The published Greece pipeline carried two minor inconsistencies in the σ_{obs} scalar; version 0.1.0 corrected both. The corrections shift TSS values slightly (typically $< 1\%$ per model) and therefore shift HMperf and the model rankings. The corrected behaviour is canonical from version 0.1.0 onward. Reference outputs from the published Greece pipeline are archived under `documentation/historical_paper_outputs/` for traceability.

Correction 1: $\cos\varphi$-weighted spatial mean. The published pipeline reduced the per-pixel std-over-months map to the σ_{obs} scalar via an *unweighted* `.mean(dim=['lat', 'lon'])`, inconsistent with the $\cos\varphi$-weighted spatial mean used everywhere else in the pipeline (per-model `std_dev`, `bias_score`, all per-pixel comparison metrics). The corrected pipeline routes the σ_{obs} spatial mean through `subselect.performance._spatial_weighted_mean`, the same $\cos\varphi$-weighted helper that every other spatial mean uses.

Correction 2: native 0.5° reference for σ_{obs}. The published pipeline read σ_{obs} from the cmip6-grid upscaled product (`Data/reference/monthly_upscaled/<var>_*_cmip6_upscaled.nc`, 1.25° × 1.875°, 144 × 192). The corrected pipeline reads from the native 0.5° single-grid product (`Data/reference/monthly_05/<var>_*.nc`, 0.5° × 0.5°, 360 × 720) via `subselect.io.load_native_w5e5`. The native product is the direct W5E5 obs without an upscaling pre-processing step, so σ_{obs} no longer carries upscaling-imprint variance.

Why these are correct. Both corrections remove confounds that should not be in the σ_{obs} scalar. (1) An unweighted spatial mean double-counts high-latitude pixels relative to their actual area; the rest of the pipeline uses $\cos\varphi$ weights for exactly this reason, and the original inconsistency was an oversight rather than a methodology choice. (2) Computing σ_{obs} on a coarser grid than the per-pixel metrics introduces resolution-dependent smoothing into the TSS denominator that has no physical basis.

Regression contract. Version 0.1.0 pins parquet snapshots of the corrected pipeline outputs (`compute_metrics` per variable, `compute_hps`, `compute_change_signals`) under `tests/fixtures/regression_corrected/`. `tests/test_regression_corrected.py` asserts bit-identity at `1e-12` against these snapshots.

Decided. 2026-05-02, in version 0.1.0.

2 Future-response spread

The future-spread dimension is partially inherited from the published Greece paper (Tsilimigkras et al., 2026) (change variables, scenario default) and partially extended for the framework (formal sub-ensemble coverage metric).

2.1 Change-signals regression-test tolerance

Question. What tolerance does the change-signals regression test use against the published xlsx, and what failure mode justifies a non-uniform tolerance across the 35-model ensemble?

Choice. "ATOL_TARGET = 1e-6" for 34/35 models; "ATOL_CHANGE_SIGNALS = 5e-4" as the test-wide tolerance to absorb a single-model float32 drift on IPSL-CM6A-LR's psl columns specifically.

Rationale. CMIP6 NetCDF files store fields as float32, which are read and promoted to float64 during xarray operations. For 34 models the promotion order matches the published Greece pipeline exactly, so the change-signal scalars round-trip bit-identically (observed max absolute diff $< 10^{-12}$). IPSL-CM6A-LR is the lone outlier: it uses a noleap calendar, and xarray's time-decoding path for non-standard calendars promotes float32 → float64 at a slightly later operation than for standard calendars. The arithmetic is identical; only the order of precision promotion differs. Catastrophic cancellation between two $\sim 10^5$ Pa pre-industrial and end-of-century pressure values amplifies the float32 ULP ($\sim$0.012 Pa) into the observed $\sim 10^{-4}$ relative error on the change signal. tas / tasmax for IPSL also drift but stay within 2×10^{-6} because cancellation is mild for $\sim$6 K change signals on

$\sim$300 K absolute values. The 5×10^{-4} tolerance leaves a $\sim$5$\times$ margin past the observed maximum breach (1.83×10^{-4} on `psl_DJF`); the spread-quadrant labels (cross-model medians on Δtas / Δpr) are unaffected because the float32 noise is orders of magnitude smaller than the inter-model spread on which the discrete labels rest.

Implementation. `tests/test_regression_greece.py::test_greece_change_signals_regression` (uses `ATOL_CHANGE_SIGNALS`). The HPS regression test stays at `ATOL_TARGET`.

Decided. 2026-05-01, in version 0.1.0.

3 Model independence

The independence dimension is new in the framework; it was not part of the published Greece paper (Tsilimigkras et al., 2026). Two methods are scoped: feature-space clustering on a regional climate-response embedding, and pairwise-RMSE genealogy clustering.

4 Multi-objective optimization

The selection engine combines the three dimensions into a scalar cost (default) and a Pareto front (diagnostic). See `subselect.optimize` for the implementation.

5 Country cropping

The framework supports both bounding-box and shapefile-based cropping of the input fields. See `subselect.geom` for the implementation.

5.1 Framework default crop rule

Question. Which cropping rule should the framework apply by default when extracting a CMIP6 model's data over a user-specified country?

Alternatives considered.

- **`bbox`**: rectangular slice from `country_codes.json`, no shapefile dependency. The published Greece paper used this; it over-includes ocean and neighbouring land, especially for archipelagic countries.
- **`shapefile_strict`**:

  binary mask via `rasterio.features.geometry_mask` with `all_touched=False` — a pixel is included only if its centroid lies inside the country polygon. Risk: leaves zero pixels for small countries on coarse grids (e.g. Cyprus on CanESM5 $\sim$2.8 °).
- **`shapefile_lenient`**: binary mask via `rasterio.features.geometry_mask` with `all_touched=True` — any pixel touched by the country polygon is included. NaN outside the country.
- **`shapefile_fractional`**: per-pixel area-fraction inside the country polygon, in $[0, 1]$, via per-cell `shapely` intersection. Returns the data un-masked plus a weight array. Boundary-precision option; opt-in.

Choice. `shapefile_lenient`.

Rationale. Three reasons. First, *operational simplicity*: the cropped result is a rectangular `DataArray` with NaN outside the country, which xarray and matplotlib both handle natively; the existing $\cos(\varphi)$ area-weighted spatial-mean code works unchanged because the `xr.DataArray.weighted(...).mean(...)` primitive already ignores NaN entries. Second, *prototype consistency*: the legacy `climpact/shp_extraction.ipynb` prototype already uses this rule, and adopting it as the default avoids re-litigating an empirical choice that was satisfactory in earlier exploratory work. Third, *empirical validation*: visual comparison of all four methods over Greece, Cyprus, and Switzerland on CanESM5 and MPI-ESM1-2-HR confirmed that `shapefile_lenient` captures the country footprint sensibly across archipelagic (Greece), small-island (Cyprus), and mountainous-landlocked (Switzerland) geometries on coarse and fine native grids. The regression test continues to pin `bbox` for parity with the published Greece numbers; `shapefile_fractional` remains available for boundary-precision use cases that justify the larger interface change to weight-aware metrics.

Definition. For each pixel p on the model's native grid with spatial extent A_p:

$$m(p) = \begin{cases} 1 & \text{if } A_p \cap \Omega \neq \emptyset, \\ 0 & \text{otherwise,} \end{cases} \tag{1}$$

where Ω is the country polygon from GADM 4.1 (layer `ADM_0`). The mask is computed against the full input grid; both the data and the mask are then cropped to a mask-aware bounding box (the smallest lat/lon box enclosing all $m(p) = 1$ pixels, padded by one full cell on each side). The cropped data is masked with NaN where $m(p) = 0$:

$$d_{\text{crop}}(p) = \begin{cases} d(p) & \text{if } m(p) = 1, \\ \text{NaN} & \text{if } m(p) = 0. \end{cases} \tag{2}$$

Implementation. `subselect.geom.crop` with `method="shapefile_lenient"` (the default kwarg value); pinned by `tests/test_geom.py::test_lenient_includes_cell_when_polygon_lands_inside_a_single_cell` and the overshoot-half-cell regression case alongside it.

Decided. 2026-04-30, in version 0.1.0.

6 Caching and reproducibility

The framework caches per-country results in two layers: parquet for tabular metric tables and zarr for fields and matrices, indexed by a small JSON catalog per scope.

6.1 Cache stack: parquet + zarr + SQLite

Question. Which storage stack should back the per-country and ensemble-wide derived artefacts (HPS / BVS tables, change signals, climatology fields, GWL crossing years, distance matrices)?

Alternatives considered.

- **Parquet (tabular) + Zarr (arrays) + SQLite (index).** Three layers, each with one responsibility. Parquet is language-portable, schema-explicit, snappy-compressed; Zarr v2 directory stores serve trivially as static files; SQLite is in the Python stdlib.
- **DuckDB on Parquet.** Same parquet files, DuckDB replaces SQLite as the catalog *and* runs analytical queries directly against the parquet files. Extra dependency.
- **Intake catalog** (`intake-xarray`). YAML-based, declarative, climate-community standard for indexing xarray datasets.
- **HDF5 hierarchies** (`h5py` or `netCDF4`). Single-file, hierarchical container.

Choice. Parquet (tabular) + Zarr (arrays) + SQLite (index).

Rationale. Four reasons. First, *workload fit*: tabular outputs are KB to MB-scale per artefact (the canonical Greece directory is ∼138 MB across 28 spreadsheets), and the hot path is per-country DataFrame loads in the Phase 2 cost-function inner loop. DuckDB's value is analytical SQL across many parquet files; that workload does not exist here, so SQLite as a pure path-catalog is the right weight class. Second, *consumer set*: the cache is read only by `subselect` itself and the future static web app; `intake-xarray`'s value is sharing a declarative catalog with third-party climate tooling, which is not a requirement. Third, *web-app delivery*: zarr v2 directory stores serve trivially over HTTP as static files (the Phase 4 web-app delivery model); HDF5 single-files require a server runtime to slice. Fourth, *reproducibility hooks*: cache invalidation by code version is a single JSON-catalog field; harder to retrofit into intake YAML or HDF5 attributes. Earlier exploratory pickle artefacts were language-locked and version-fragile; parquet replaces them and removes that dependency.

Definition. The cache is per-country and indexed by a small JSON catalog at `cache/`$\langle c\rangle$`/catalog.json`. Each artefact records its on-disk path, the maximum mtime of its declared inputs at write time, and the time of computation. Layout:

tabular	cache/$\langle c\rangle$/`parquet/`$\langle k\rangle$`.parquet`
dict-tabular	cache/$\langle c\rangle$/`parquet/`$\langle k\rangle$/$\langle s\rangle$`.parquet`
dataclass	cache/$\langle c\rangle$/`parquet/`$\langle k\rangle$/ $\{\langle f\rangle$`.parquet`, `scalars.json`$\}$
fields	cache/$\langle c\rangle$/`zarr/`$\langle k\rangle$`.zarr`

where c = country, k = artefact key, s = sub-key, f = dataclass field. On read, an artefact is invalidated whenever any declared input file has an mtime newer than the recorded maximum.

Implementation. `subselect.cache.Cache` exposes `has`, `is_fresh`, `save`, `load`, `invalidate`, `clear`. Artefact kinds are inferred from value type (`DataFrame`, `Series`, `Dataset`, `DataArray`, `dict[str, DataFrame]`, dataclass). `zarr<3` pinned in `environment.yml` and `pyproject.toml` for v2 directory-store stability.

Decided. 2026-05-01, in version 0.1.0. An earlier draft of the cache layer carried a SQLite catalog; the simpler scope (one country, one cache, one catalog) made cross-country indexing unnecessary and the per-scope JSON catalog used in version 0.1.0 onward removes SQLite's I/O and schema overhead.

Cache scope: global vs per-country

Question. Where do per-(model, variable) climatology fields, σ_{obs} grids, and per-(model, variable, scenario) annual-mean fields live?

Choice. Country-independent artefacts move to `cache/_global/`; country-specific reductions stay under `cache/`$\langle c\rangle$`/`. The *compute-once-globally, reuse-per-country* convention.

Rationale. A per-(model, variable) monthly climatology over the 1995–2014 evaluation window depends only on (model, variable), not on the country chosen for analysis. The same is true of the corresponding upscaled-observation climatology, the per-period interannual σ map, the end-of-century and pre-industrial monthly climatologies, and the per-(model, variable, scenario) annual-mean field 1850–2100. Country analysis cropping is a slice of the cached field, not a recomputation of it. Caching at country scope would re-pay the per-model climatology and annual-mean cost on every new country; caching at global scope pays it once. After the first country run, every subsequent country reuses the cached fields.

Layout.

global	cache/_global/{catalog.json, zarr/$\langle a \rangle$.zarr, parquet/$\langle k \rangle$.parquet}
per-country	cache/$\langle c \rangle$/{catalog.json, parquet/, zarr/}

Global artefacts.

- Per-(model, variable) historical climatology (historical_clim__<var>__<model>.zarr); 12 months on the model's native grid over 1995–2014.
- Per-(model, variable) upscaled-observation climatology (obs_clim__<var>__<model>.zarr).
- Per-(model, variable, period) interannual σ map (sigma_ref__<var>__<model>.zarr).
- Per-(model, variable, scenario) end-of-century climatology (eoc_clim__<var>__<model>__<ssp>.zarr); 12 months over 2081–2100.
- Per-(model, variable) pre-industrial climatology (pi_clim__<var>__<model>.zarr); 12 months over 1850–1899.
- Per-(model, variable, scenario) annual-mean field (annual_field__<var>__<model>__<ssp>.zarr); 251 years on the model's native grid.
- Native 0.5° W5E5 climatology + per-period σ field per variable (native_sigma_obs__<var>.zarr); feeds the σ_{obs} scalar in the TSS denominator.

Per-country artefacts are country-mean reductions of the above: HPS / TSS / BVS metric tables, country annual time-series, change_signals, country σ_{obs} scalars per period, country profile signals, country-mean monthly climatologies, country-mean bias fields, and the warming-level / future-anomaly tables.

Implementation. subselect.compute_global.compute_global() populates the global cache; subselect.compute.compute(country) auto-builds the global cache on first call and consumes it thereafter. The Cache class' global_cache classmethod wraps the global scope. The CLI exposes python -m subselect -global-only for explicit warmup and -force {all,country,global} for scope-targeted recompute.

Numerical artefact of the cache architecture. Compared to an earlier development draft that performed *load* → *crop* → *time-slice* → *climatology* → *spatial-mean* per-country, the global-cache architecture used from version 0.1.0 onward computes the climatology globally and then crops: *climatology (global)* → *crop* → *spatial-mean*. The two operation orders are mathematically equivalent on a continuous-valued field; on float32 reductions the spread-pipeline outputs (change_signals, long_term_spread, pre_industrial_spread) drift by $\sim 10^{-5}$ absolute / 10^{-7} relative — six significant digits, six orders of magnitude below any climate-relevant threshold. This is a numerical artefact of the cache architecture, not a methodology change. The post-cache values are the canonical regression-test contract.

Cross-scope invalidation. Per-country artefacts that derive from global-cache contents include cache/_global/catalog.json in their dependency list. Saves into the global cache rewrite this catalog (its mtime advances), so the next is_fresh check on any downstream per-country artefact returns False and triggers a recompute. This prevents the failure mode where a global-cache implementation change or partial rebuild leaves stale per-country derivations: any global mutation now propagates invalidation downstream. The rule applies to observed_std_dev, performance_metrics_<var>, monthly_means_<var>_*, composite_hps, composite_hps_full, change_signals, long_term_spread, pre_industrial_spread, and annual_timeseries_<var>. A unit test in tests/test_cache.py::test_cross_scope_invalidation pins the contract.

Performance footprint. Cold global compute: ∼5–7 min (one-time per machine; 35 models × 4 vars × 4 SSPs NetCDF reads producing 1380 zarr entries totalling ∼3 GB). Warm global + cold country: ∼1:45 (Greece), 1:42 (Sweden), 1:56 (Belgium). Warm cache (every artefact cached, 28 figures rendered): ∼50 s, dominated by cartopy bias-map rendering. Further optimisation (batched zarr reads, parallel figure render, bias-map persistence) is a future performance sprint.

Decided. 2026-05-02, in version 0.1.0.

References

A. Tsilimigkras, M. Lazaridis, A. Voulgarakis, et al. Climate projections for Greece: Defining a regional sub-ensemble from the CMIP6 landscape. *Theoretical and Applied Climatology*, 157:123, 2026. doi: 10.1007/s00704-026-06029-w. URL `https://doi.org/10.1007/s00704-026-06029-w`.